

Oracle EAM Table Relational Map

Schema: APPS | Org: 1169 | Org Code: XAU

Table Index

Table	Schema	Type	Purpose
EAM_WORK_ORDERS_V	APPS	View	Work order headers. The central hub — one row per work order. Carries status, type, asset, activity, and schedule. Built on top of WIP_DISCRETE_JOBS with EAM-specific columns added.
QA_RESULTS	APPS	Synonym → QA schema	Quality inspection results. Each row is a single inspection entry submitted against a work order, keyed to a collection plan. Stores lot, location, reason, adjustment, and free-text description in CHARACTER1-4 and COMMENT1 .
WIP_OPERATIONS_V	APPS	View	Work order operation steps. Each work order can have multiple numbered operations (steps). Tracks whether each step is complete and which department owns it. OPERATION_SEQ_NUM = 10 is the standard first/primary step in most EAM scripts here.
WIP_OP_RESOURCE_INSTANCES_V	APPS	View	Resource assignments per operation step. Records which specific equipment or machine instance (INSTANCE_NAME) is assigned to a given operation on a work order. Joins via both WIP_ENTITY_ID and OPERATION_SEQ_NUM .
MTL_PARAMETERS	APPS	Synonym → INV schema	Inventory organization config. One row per organization. Used here primarily to translate between numeric ORGANIZATION_ID (1169) and human-readable ORGANIZATION_CODE (XAU). Gateway for all org-scoped queries.
FND_USER	APPS	Synonym → APPLSYS schema	Application user accounts. Every Oracle EBS login. Links numeric USER_ID (used in audit columns like QA_CREATED_BY) to a readable USER_NAME (typically the employee number) and to the HR person record via EMPLOYEE_ID .
PER_ALL_PEOPLE_F	APPS	Synonym → HR schema	HR person master (date-effective). One row per effective period per person. Used to resolve a USER_ID to a FULL_NAME for display. Must always be filtered with SYSDATE BETWEEN EFFECTIVE_START_DATE AND EFFECTIVE_END_DATE to avoid duplicates.
EAM_PM_SCHEDULING_RULES	EAM	Table	Preventive maintenance scheduling rules. Defines the triggers and frequencies for PM work order auto-generation. Has 42,213 rows — including it in a FROM clause without a join condition causes a Cartesian product.
FND_USER_RESP_GROUPS_DIRECT	APPS	Synonym → APPLSYS schema	User-to-responsibility assignments. Junction table that records which responsibilities a user has been directly granted, with start/end dates. Used to answer "who has the EAM Manager role?"
FND_RESPONSIBILITY	APPS	Synonym → APPLSYS schema	Responsibility definitions. A responsibility in EBS is a named set of menus, functions, and data access rules. Joins to FND_APPLICATION for the module (e.g. EAM) and to FND_RESPONSIBILITY_TL for the display name.
FND_RESPONSIBILITY_TL	APPS	Synonym → APPLSYS schema	Translated responsibility names. Stores the human-readable responsibility name per installed language. Always filter LANGUAGE = 'US' — without it you get one row per installed language (14 languages confirmed in this instance).
FND_APPLICATION	APPS	Synonym → APPLSYS schema	Oracle application/module registry. One row per EBS module. Used to scope responsibility lookups to a specific module via APPLICATION_SHORT_NAME (e.g. 'EAM').
HR_ALL_ORGANIZATION_UNITS	HR	Table	HR organization master — Business Group root. Every HR record partitions back to this table via BUSINESS_GROUP_ID . In EAM context this is the top-level org container, not a table you query directly. FK expansion stops here — going deeper maps the full HRMS/Payroll suite.
QA_USER_GROUP_V	APPS	View	QA user group membership. Shows which QA groups a user belongs to, with status and party/relationship identifiers. Already includes USER_NAME , EMAIL_ADDRESS , and PERSON_NAME — no join to FND_USER or PER_ALL_PEOPLE_F needed for basic lookups. Also joins directly to QA_RESULTS via QA_CREATED_BY to enrich inspection results with group membership.
WIP_ENTITIES	APPS	Synonym → WIP schema	Work order base table. The raw entity behind EAM_WORK_ORDERS_V . Carries standard EBS audit columns (CREATED_BY , LAST_UPDATED_BY) not always exposed by the view. Used for audit trail queries.
WIP_OPERATIONS	APPS	Synonym → WIP schema	Work order operations base table. The raw table behind WIP_OPERATIONS_V . Carries LAST_UPDATED_BY for audit trail queries tracking who modified an operation step.
BOM_DEPARTMENTS	BOM	Table	Department master. Resolves the numeric DEPARTMENT_ID on WIP_OPERATIONS to a readable DEPARTMENT_CODE and description. Departments define which group owns or performs a work order operation. Join on both DEPARTMENT_ID and ORGANIZATION_ID .

Table	Schema	Type	Purpose
BOM_RESOURCES	BOM	Table	Resource definitions. One row per resource (person, machine, or equipment) per org. RESOURCE_TYPE = 2 = Person. RESOURCE_CODE is the display name seen in the EAM Resources form. DISABLE_DATE IS NULL = active.
BOM_RESOURCE_EMPLOYEES	BOM	Table	Employee-to-resource assignments. Junction table linking HR persons to BOM resources. Join via PERSON_ID directly to PER_ALL_PEOPLE_F — no FND_USER needed. EFFECTIVE_END_DATE controls when the assignment expires.
MTL_MATERIAL_TRANSACTIONS	INV	Table	Inventory material transactions. Every inventory movement — issues, receipts, returns. Filter TRANSACTION_SOURCE_TYPE_ID = 5 for WIP-sourced transactions. Join to EAM_WORK_ORDERS_V via TRANSACTION_SOURCE_ID = WIP_ENTITY_ID to get parts charged to a work order. Negative qty = issued out, positive = returned.
MTL_SYSTEM_ITEMS_B	INV	Table	Item/part master. One row per item per org. SEGMENT1 is the part number. Join via INVENTORY_ITEM_ID and ORGANIZATION_ID.
MTL_TRANSACTION_TYPES	INV	Table	Transaction type lookup. Resolves TRANSACTION_TYPE_ID to a human-readable name (e.g. WIP Issue, WIP Return, WIP Scrap).

EBS Synonym Pattern: Most APPS.* objects are synonyms pointing to base tables in other schemas (QA , INV , APPLSYS , HR , etc.). Always use APPS.tablename in queries — the synonym works fine. But when querying ALL_TAB_COLUMNS or ALL_OBJECTS to inspect metadata, you must resolve the base schema first via ALL_SYNONYMS .

Discovered via FK scan (one level out)

Table	Schema	Type	Discovered Via
HR_ALL_ORGANIZATION_UNITS	HR	Table	PER_ALL_PEOPLE_F.BUSINESS_GROUP_ID → HR_ALL_ORGANIZATION_UNITS.ORGANIZATION_ID (constraint: PER_PEOPLE_F_FK1)
BOM_RESOURCES	BOM	Table	Resource definitions (people, machines, equipment) — discovered via EAM Resources form. RESOURCE_TYPE = 2 = Person
BOM_RESOURCE_EMPLOYEES	BOM	Table	PER_ALL_PEOPLE_F.PERSON_ID → BOM_RESOURCE_EMPLOYEES.PERSON_ID — links employees directly to resources, no FND_USER needed
BOM_DEPARTMENTS	BOM	Table	WIP_OPERATIONS.DEPARTMENT_ID → BOM_DEPARTMENTS.DEPARTMENT_ID — resolves department ID to readable code and description
MTL_MATERIAL_TRANSACTIONS	INV	Table	EAM_WORK_ORDERS_V.WIP_ENTITY_ID → MTL_MATERIAL_TRANSACTIONS.TRANSACTION_SOURCE_ID (filter: TRANSACTION_SOURCE_TYPE_ID = 5) — all parts issued/returned against a work order
MTL_SYSTEM_ITEMS_B	INV	Table	MTL_MATERIAL_TRANSACTIONS.INVENTORY_ITEM_ID → MTL_SYSTEM_ITEMS_B.INVENTORY_ITEM_ID — part number and description
MTL_TRANSACTION_TYPES	INV	Table	MTL_MATERIAL_TRANSACTIONS.TRANSACTION_TYPE_ID → MTL_TRANSACTION_TYPES.TRANSACTION_TYPE_ID — human-readable transaction type name

FK expansion stopped here. Layer 2 scan of HR ALL ORGANIZATION_UNITS found ~60 additional HR schema tables that all reference it via BUSINESS_GROUP_ID — this is the Business Group root for Oracle HRMS. Expanding further would map the entire HR/Payroll module. See the HR_ALL_ORGANIZATION_UNITS section below for the short list of EAM-relevant child tables.

Table Details

APPS.EAM_WORK_ORDERS_V

Purpose: A view over WIP_DISCRETE_JOBS that surfaces EAM-specific columns alongside standard WIP fields. Think of it as the "work order header" — one row per work order, carrying who owns it, what asset it's for, what activity, what status, and when it was scheduled. Every other table in this map joins back to it. **Primary Key:** WIP_ENTITY_ID **Hub table — nearly every query joins through here.**

Column	Notes
WIP_ENTITY_ID	PK — joined from QA_RESULTS, WIP_OPERATIONS_V, WIP_OP_RESOURCE_INSTANCES_V
WIP_ENTITY_NAME	Work order number (e.g. AU%, PM%)
ORGANIZATION_ID	FK → MTL_PARAMETERS
WORK_ORDER_STATUS	e.g. Released , Cancelled

Column	Notes
USER_DEFINED_STATUS_ID	Numeric status (e.g. 3 = Released in level10 query)
WORK_ORDER_TYPE_DISP	e.g. DM (demand maintenance)
ASSET_NUMBER	Equipment identifier (e.g. AU%)
ASSET_ACTIVITY	PM activity code
ASSET_DESCRIPTION	Asset description text
ASSET_GROUP_ID	Asset classification group
CLASS_CODE	Work order class
DESCRIPTION	Work order description
CREATION_DATE	Work order creation timestamp
SCHEDULED_START_DATE	Planned start date

APPS.QA_RESULTS

Purpose: Stores the results of quality collection plans executed during work order operations. When a technician fills out an inspection form in EAM (lot number, location, reason for adjustment, etc.), that submission lands here. Each row ties back to a work order via `WORK_ORDER_ID` and to a specific operation step via `MAINTENANCE_OP_SEQ`. The generic `CHARACTER1-4` and `COMMENT1` columns carry the actual user-entered values — their meaning depends on the collection plan. **Join to EAM_WORK_ORDERS_V:** `QR.WORK_ORDER_ID = EWO.WIP_ENTITY_ID AND QR.ORGANIZATION_ID = EWO.ORGANIZATION_ID`

Column	Notes
WORK_ORDER_ID	FK → EAM_WORK_ORDERS_V.WIP_ENTITY_ID
ORGANIZATION_ID	FK → MTL_PARAMETERS / EAM_WORK_ORDERS_V
QA_CREATED_BY	FK → FND_USER.USER_ID
QA_LAST_UPDATED_BY	FK → FND_USER.USER_ID
QA_CREATION_DATE	When the QA record was created
QA_LAST_UPDATE_DATE	Last update timestamp
MAINTENANCE_OP_SEQ	Operation sequence (filter: = 10)
PLAN_ID	QA collection plan ID
COLLECTION_ID	QA collection ID
OCCURRENCE	Occurrence number within collection
STATUS	QA result status
CHARACTER1	→ Lot Number
CHARACTER2	→ Location
CHARACTER3	→ Reason
CHARACTER4	→ Adjustment
COMMENT1	→ Description / notes

APPS.WIP_OPERATIONS_V

Purpose: Each work order is broken into one or more numbered operation steps. This view exposes those steps — which department is responsible, whether the step is complete, and what sequence number it is. In all current EAM scripts here, only sequence 10 (the first/primary step) is used. Some queries alias this view twice (`W01` , `W02`) to simultaneously check two different operation sequences on the same work order. **Join to EAM_WORK_ORDERS_V:** `WO.WIP_ENTITY_ID = EWO.WIP_ENTITY_ID` **Join to WIP_OP_RESOURCE_INSTANCES_V:** `WO.WIP_ENTITY_ID + WO.OPERATION_SEQ_NUM`

Column	Notes
WIP_ENTITY_ID	FK → EAM_WORK_ORDERS_V

Column	Notes
ORGANIZATION_ID	FK → MTL_PARAMETERS
OPERATION_SEQ_NUM	Step number (filter: = 10)
OPERATION_COMPLETED	Y / N flag
DEPARTMENT_CODE	Dept responsible for the operation

Note: Some queries alias this view twice (W01 , W02) to check multiple operation seq numbers independently on the same work order.

APPS.WIP_OP_RESOURCE_INSTANCES_V

Purpose: Tracks which specific equipment or machine instance is assigned to a work order operation step. In a mining/asset context like XAU, INSTANCE_NAME typically identifies the physical piece of equipment (e.g. a specific truck or drill) that will perform or be the subject of the work. Requires both WIP_ENTITY_ID and OPERATION_SEQ_NUM to join correctly — the operation seq is the disambiguator when a work order has multiple steps. **Join:** RES.WIP_ENTITY_ID = EWO.WIP_ENTITY_ID AND RES.OPERATION_SEQ_NUM = W02.OPERATION_SEQ_NUM

Column	Notes
WIP_ENTITY_ID	FK → EAM_WORK_ORDERS_V
OPERATION_SEQ_NUM	FK → WIP_OPERATIONS_V
INSTANCE_NAME	Resource/equipment instance name

APPS.MTL_PARAMETERS

Purpose: The configuration record for an inventory organization in EBS. One row per org. In EAM queries it serves a single purpose: translating the numeric ORGANIZATION_ID (1169) to the human-readable ORGANIZATION_CODE (XAU), or vice versa. Can also be used to confirm that a given ID/code pair is consistent before using either as a filter. **Join:** MP.ORGANIZATION_ID = EWO.ORGANIZATION_ID **Used to filter by:** ORGANIZATION_CODE = 'XAU'

Column	Notes
ORGANIZATION_ID	PK — links to all org-scoped tables
ORGANIZATION_CODE	Human-readable org code (e.g. XAU)

APPS.FND_USER

Purpose: The EBS application user account table. Every person who can log into Oracle EBS has a row here. In EAM queries it bridges audit columns (like QA_CREATED_BY , which stores a numeric USER_ID) to readable names and HR records. The USER_NAME is typically the employee's badge/payroll number. The EMPLOYEE_ID links to PER_ALL_PEOPLE_F to get the full name. **Join to QA_RESULTS:** FU.USER_ID = QR.QA_CREATED_BY or QR.QA_LAST_UPDATED_BY **Join to PER_ALL_PEOPLE_F:** FU.EMPLOYEE_ID = PEO.PERSON_ID **Join to FND_USER_RESP_GROUPS_DIRECT:** FU.USER_ID = FUR.USER_ID

Column	Notes
USER_ID	PK — matched to QA_RESULTS created/updated by fields; also join key for responsibility tables
USER_NAME	Login name — often an employee number (e.g. '10169062')
DESCRIPTION	User's full name (used as display name in user queries)
EMAIL_ADDRESS	User email
EMPLOYEE_ID	FK → PER_ALL_PEOPLE_F.PERSON_ID
END_DATE	Account expiry — filter IS NULL OR > SYSDATE for active users

User lookup tip: A user can be found by USER_NAME , USER_ID , or EMPLOYEE_ID — all three may carry the same employee number. Use TO_CHAR() when comparing numeric IDs to string input.

Triple-OR lookup pattern — find a user when you don't know which identifier you have: sql WHERE fu.user_name = '10169062' OR TO_CHAR(fu.user_id) = '10169062' OR TO_CHAR(fu.employee_id)

= '10169062' Use LEFT JOINs to FND_USER_RESP_GROUPS_DIRECT → FND_RESPONSIBILITY → FND_RESPONSIBILITY_TL → FND_APPLICATION to get all responsibilities assigned to that user.

APPS.PER_ALL_PEOPLE_F

Purpose: The HR person master — the source of truth for employee names and identities. The `_F` suffix means it's date-effective (slowly changing dimension): a person can have multiple rows covering different time periods as their details change. Used in EAM queries solely to resolve a `USER_ID` → `EMPLOYEE_ID` → `FULL_NAME` for display purposes. The `BUSINESS_GROUP_ID` column is an FK to `HR_ALL_ORGANIZATION_UNITS` which is the top-level HR partition boundary.
Join: `FU.EMPLOYEE_ID = PEO.PERSON_ID` **Date filter required:** `SYSDATE BETWEEN PEO.EFFECTIVE_START_DATE AND PEO.EFFECTIVE_END_DATE`

Column	Notes
PERSON_ID	PK
FULL_NAME	Employee display name
EFFECTIVE_START_DATE	Date-effectivity start
EFFECTIVE_END_DATE	Date-effectivity end
BUSINESS_GROUP_ID	FK → HR.HR_ALL_ORGANIZATION_UNITS.ORGANIZATION_ID (constraint: PER_PEOPLE_F_FK1)

Always include the date range filter — this is a slowly changing dimension table and will return duplicates without it.

HR.HR_ALL_ORGANIZATION_UNITS

Discovered via FK scan. Parent table referenced by `PER_ALL_PEOPLE_F.BUSINESS_GROUP_ID`. **APPS synonym:** `APPS.HR_ALL_ORGANIZATION_UNITS`

Column	Notes
ORGANIZATION_ID	PK — referenced by <code>PER_ALL_PEOPLE_F.BUSINESS_GROUP_ID</code>
NAME	Organization / business group name
DATE_FROM	Effectivity start date
DATE_TO	Effectivity end date (NULL = still active)

Business Group root — natural stopping point for FK expansion. In EBS, a Business Group is the top-level HR partition. Almost every table in the HR schema (`PER_*`, `PAY_*`, `PQH_*`) carries `BUSINESS_GROUP_ID` as an FK back to this table. The second-layer FK scan confirmed ~60 HR module tables reference it — `PER_ALL_ASSIGNMENTS_F`, `PAY_ALL_PAYROLLS_F`, `PER_GRADES`, `PER_JOBS`, `PER_PERIODS_OF_SERVICE`, and many others. Expanding further would pull in the entire Oracle HRMS/Payroll suite, which is outside the scope of EAM work order queries. This table is documented here for completeness; **do not join to it in EAM queries** — the `BUSINESS_GROUP_ID` on `PER_ALL_PEOPLE_F` is a system partition key, not a field you need to filter on.

EAM-relevant child tables discovered at layer 2 (for reference only — these are HR module tables, not used in current EAM scripts):

Table	Schema	Relevance
PER_ALL_ASSIGNMENTS_F	HR	Links employees to org units, jobs, positions — could be used to look up a person's assignment from their <code>PERSON_ID</code>
PER_PERIODS_OF_SERVICE	HR	Employment start/end dates — useful for active employee checks
HR_ORGANIZATION_INFORMATION	HR	Extended org attributes keyed by <code>ORGANIZATION_ID</code>
PER_JOBS	HR	Job definitions referenced by assignments

All other ~55 tables discovered at this layer (`PAY_*`, `PQH_*`, `PER_COBRA_*`, `PER_ELECTION_*`, etc.) are payroll, benefits, and HR admin tables with no current relevance to EAM work order reporting.

EAM.EAM_PM_SCHEDULING_RULES

Purpose: Defines the rules that drive automatic Preventive Maintenance work order generation — things like "create a work order every 250 hours of runtime" or "create one every 30 days". Each row is a scheduling rule tied to a PM activity. The `SCHEDULED_START_DATE` column appears in the `pm_released_work_orders.sql` `SELECT` but the table is included in the `FROM` clause with no join condition, which causes a Cartesian product because this table has 42,213 rows. Used in `pm_released_work_orders.sql` with no explicit join condition (Cartesian).

Column	Notes
<code>SCHEDULED_START_DATE</code>	PM scheduled start date — appears in <code>SELECT</code>

⚠ Confirmed Cartesian Risk: Verification run confirmed this table has **42,213 rows**. The existing `pm_released_work_orders.sql` includes it in the `FROM` clause with no join condition, which will multiply every result row by 42,213. This script needs an explicit join condition added before use in production.

Security & User Management Tables

APPS.FND_USER_RESP_GROUPS_DIRECT

Purpose: The junction table that records which responsibilities a user has been explicitly granted. Each row says "user X has been given responsibility Y, starting on date Z". Used to answer questions like "who currently has the ICU EAM Manager - XAU role?" — joining from `FND_USER` through here to `FND_RESPONSIBILITY` and `FND_RESPONSIBILITY_TL`. **Join to FND_USER:** `FUR.USER_ID = FU.USER_ID` **Join to FND_RESPONSIBILITY:** `FUR.RESPONSIBILITY_ID = FR.RESPONSIBILITY_ID`

Column	Notes
<code>USER_ID</code>	FK → <code>FND_USER.USER_ID</code>
<code>RESPONSIBILITY_ID</code>	FK → <code>FND_RESPONSIBILITY</code> / <code>FND_RESPONSIBILITY_TL</code>
<code>START_DATE</code>	When the responsibility was assigned
<code>END_DATE</code>	When it expires — check for NULL or future date for active assignments

This is the **direct** assignment table. Oracle also has `FND_USER_RESP_GROUPS_ALL` which includes inherited/indirect assignments. Use `_DIRECT` when you want only explicit grants.

APPS.FND_RESPONSIBILITY

Purpose: The definition of an EBS responsibility. A responsibility is a named access profile that determines what menus, functions, and data a user can see. It belongs to one application module (via `APPLICATION_ID`) and can have many users assigned to it. This table holds the IDs and module linkage — the human-readable name is in `FND_RESPONSIBILITY_TL`. **Join to FND_USER_RESP_GROUPS_DIRECT:** `FR.RESPONSIBILITY_ID = FUR.RESPONSIBILITY_ID` **Join to FND_APPLICATION:** `FR.APPLICATION_ID = FA.APPLICATION_ID` **Join to FND_RESPONSIBILITY_TL:** `FR.RESPONSIBILITY_ID = FRT.RESPONSIBILITY_ID`

Column	Notes
<code>RESPONSIBILITY_ID</code>	PK
<code>APPLICATION_ID</code>	FK → <code>FND_APPLICATION.APPLICATION_ID</code>

APPS.FND_RESPONSIBILITY_TL

Purpose: The translated name table for responsibilities. The `TL` suffix is Oracle's standard pattern for multi-language support — the same responsibility has one row per installed language. This instance has 14 languages installed. When querying for responsibility names, always join here (not to `FND_RESPONSIBILITY` directly for the name) and always filter `LANGUAGE = 'US'` to get exactly one row per responsibility. **Join:** `FRT.RESPONSIBILITY_ID = FR.RESPONSIBILITY_ID` AND `FRT.LANGUAGE = 'US'` **Always filter:** `LANGUAGE = 'US'` to avoid duplicate rows per language

Column	Notes
<code>RESPONSIBILITY_ID</code>	FK → <code>FND_RESPONSIBILITY</code>

Column	Notes
RESPONSIBILITY_NAME	Human-readable name (e.g. 'ICU EAM Manager - XAU')
LANGUAGE	Language code — always filter = 'US'

TL = Translated. This is Oracle's standard pattern for multi-language name tables. Without the LANGUAGE = 'US' filter you get one row per installed language.

APPS.FND_APPLICATION

Purpose: The registry of Oracle EBS application modules — one row per module. Each module has a short name (e.g. EAM for Enterprise Asset Management, INV for Inventory, HR for Human Resources). Used in responsibility queries to scope results to a specific module, ensuring you only retrieve EAM responsibilities and not identically-named ones from other modules. **Join:** FA.APPLICATION_ID = FR.APPLICATION_ID

Column	Notes
APPLICATION_ID	PK
APPLICATION_SHORT_NAME	e.g. 'EAM' — used to scope responsibilities to a module

WIP Base Tables

These are the raw tables behind the views used in most EAM queries. You generally query them directly only when you need audit columns (CREATED_BY , LAST_UPDATED_BY) not exposed by the views.

APPS.WIP_ENTITIES

Purpose: The base work order table that EAM_WORK_ORDERS_V is built on. Every work order — whether EAM, discrete, or repetitive — has a row here. The view adds EAM-specific columns on top. Use this table directly when you need standard EBS audit columns to trace who created or last modified a work order. **APPS synonym:** APPS.WIP_ENTITIES

Column	Notes
WIP_ENTITY_ID	PK — same as EAM_WORK_ORDERS_V.WIP_ENTITY_ID
WIP_ENTITY_NAME	Work order number
ORGANIZATION_ID	FK → MTL_PARAMETERS
CREATED_BY	FK → FND_USER.USER_ID — who created the work order
CREATION_DATE	When the work order was created
LAST_UPDATED_BY	FK → FND_USER.USER_ID — who last modified it
LAST_UPDATE_DATE	When it was last modified

APPS.WIP_OPERATIONS

Purpose: The base operations table that WIP_OPERATIONS_V is built on. One row per operation step per work order. Use this table directly when you need LAST_UPDATED_BY to trace who modified an operation — the view does not always expose this column. **APPS synonym:** APPS.WIP_OPERATIONS

Column	Notes
WIP_ENTITY_ID	FK → WIP_ENTITIES / EAM_WORK_ORDERS_V
OPERATION_SEQ_NUM	Step number
ORGANIZATION_ID	FK → MTL_PARAMETERS
LAST_UPDATED_BY	FK → FND_USER.USER_ID — who last touched this step
LAST_UPDATE_DATE	When the operation was last modified
CREATED_BY	FK → FND_USER.USER_ID
CREATION_DATE	When the operation was created

BOM.BOM_DEPARTMENTS

Purpose: The department master table — one row per department per organization. Departments in EBS define which group (crew, team, cost center) owns or performs a work order operation. WIP_OPERATIONS stores only the numeric DEPARTMENT_ID ; join here to get the human-readable DEPARTMENT_CODE and description. Always join on both DEPARTMENT_ID and ORGANIZATION_ID — department codes are org-scoped and the same ID can mean different things in different orgs. **APPS synonym:** APPS.BOM_DEPARTMENTS

Column	Notes
DEPARTMENT_ID	PK — FK from WIP_OPERATIONS.DEPARTMENT_ID
DEPARTMENT_CODE	Human-readable department name shown in EAM UI
ORGANIZATION_ID	Scopes the department to an org — always include in join
DESCRIPTION	Optional description of the department
DISABLE_DATE	NULL = active; populated = inactive
DEPARTMENT_CLASS_CODE	Classification grouping for departments
MAINT_COST_CATEGORY	Maintenance cost category — EAM-specific
LOCATION_ID	Physical location linked to this department

Always join on both columns: BD.DEPARTMENT_ID = WO.DEPARTMENT_ID AND BD.ORGANIZATION_ID = WO.ORGANIZATION_ID

APPS.QA_USER_GROUP_V

Purpose: A view that exposes QA user group membership — which groups a user belongs to and whether that membership is active. In Oracle QA, user groups control access to collection plans and inspection results. The view is self-contained for user identification: it already carries USER_NAME , EMAIL_ADDRESS , PERSON_NAME , PERSON_FIRST_NAME , and PERSON_LAST_NAME , so you rarely need to join elsewhere just to display who a user is. The RELATIONSHIP_ID , SUBJECT_ID , and OBJECT_ID columns reflect the underlying HZ party relationship model that Oracle uses to link users to groups. **Join to FND_USER:** QUG.USER_ID = FU.USER_ID (optional — view already has USER_NAME) **Join to QA_RESULTS:** QUG.USER_ID = QR.QA_CREATED_BY — enriches inspection results with group membership, no intermediate table needed

Column	Notes
USER_ID	FK → FND_USER.USER_ID — join key
USER_NAME	EBS login name (typically employee number)
EMAIL_ADDRESS	User email
PARTY_ID	HZ party ID for the user
PARTY_NUMBER	Human-readable party number
PERSON_NAME	Full name — use this instead of joining PER_ALL_PEOPLE_F
PERSON_FIRST_NAME	First name
PERSON_LAST_NAME	Last name
GROUP_NAME	Name of the QA user group
STATUS	Membership status — filter = 'A' for active
VALIDATED_FLAG	Whether the group membership has been validated
RELATIONSHIP_ID	HZ party relationship ID backing this membership
SUBJECT_ID	HZ subject party (the user)
OBJECT_ID	HZ object party (the group)
REL_PARTY_ID	Relationship party ID
LAST_UPDATE_DATE	When the membership was last changed

Tip: To find all members of a QA group: WHERE GROUP_NAME = 'your group' . To find all groups a user belongs to by name: WHERE USER_NAME = '10169062' . To look up by numeric ID: WHERE USER_ID = 105240 . The view handles all three directions cleanly. Add LAST_UPDATE_DATE to the SELECT to see when a membership was last changed. Group names appear to follow an org-prefix convention — XAU PM% returns all

PM-related groups for the XAU org, suggesting a naming pattern of {ORG_CODE} {TYPE} {description}.
Use `UPPER(GROUP_NAME) LIKE '{ORG}%'` to scope group lookups to a specific organization.

Resource Assignment Tables

BOM.BOM_RESOURCES

Purpose: Defines all resources available in EBS — people, equipment, and machines. Each resource has a `RESOURCE_TYPE` (person, machine, etc.), a `RESOURCE_CODE` (the display name seen in the EAM Resources form), and an `ORGANIZATION_ID`. In EAM, resources of type Person are the ones linked to employees. The `DISABLE_DATE` indicates when a resource was deactivated. **APPS synonym:** `APPS.BOM_RESOURCES`

Column	Notes
<code>RESOURCE_ID</code>	PK — joined from <code>BOM_RESOURCE_EMPLOYEES</code>
<code>RESOURCE_CODE</code>	Human-readable resource name shown in EAM UI
<code>ORGANIZATION_ID</code>	Scopes the resource to an org — filter to 1169 for XAU
<code>DESCRIPTION</code>	Optional description of the resource
<code>RESOURCE_TYPE</code>	Lookup code — 2 = Person, 1 = Machine, 3 = Space
<code>DISABLE_DATE</code>	NULL = active; populated = inactive
<code>UNIT_OF_MEASURE</code>	e.g. HR for hour-based labor resources

BOM.BOM_RESOURCE_EMPLOYEES

Purpose: The junction table that links employees (persons) to resources. When you click the **Employees** button on the EAM Resources form, the records shown come from this table. Uses `PERSON_ID` directly — no need to go through `FND_USER`. The `EFFECTIVE_START_DATE` / `EFFECTIVE_END_DATE` dates control the active window for the assignment. **APPS synonym:** `APPS.BOM_RESOURCE_EMPLOYEES`

Column	Notes
<code>RESOURCE_ID</code>	FK → <code>BOM_RESOURCES.RESOURCE_ID</code>
<code>ORGANIZATION_ID</code>	FK — scope to org 1169
<code>PERSON_ID</code>	FK → <code>PER_ALL_PEOPLE_F.PERSON_ID</code> (direct link, no <code>FND_USER</code> needed)
<code>EFFECTIVE_START_DATE</code>	When the employee was assigned to this resource
<code>EFFECTIVE_END_DATE</code>	When the assignment ends — filter for active: <code>SYSDATE <= EFFECTIVE_END_DATE</code>
<code>INSTANCE_ID</code>	Links to a specific equipment instance if applicable

Shorter join path for employee lookups: When querying resource assignments, join `PER_ALL_PEOPLE_F` directly to `BOM_RESOURCE_EMPLOYEES` via `PERSON_ID` — you do not need `FND_USER` in the middle.

```

EAM_WORK_ORDERS_V (hub)
├── QA_RESULTS
│   ├── ON QR.WORK_ORDER_ID = EWO.WIP_ENTITY_ID
│   └── AND QR.ORGANIZATION_ID = EWO.ORGANIZATION_ID
│       ├── FND_USER (QA_CREATED_BY)
│       │   ├── ON FU.USER_ID = QR.QA_CREATED_BY
│       │   │   ├── PER_ALL_PEOPLE_F
│       │   │   │   ├── ON PEO.PERSON_ID = FU.EMPLOYEE_ID
│       │   │   │   └── AND SYSDATE BETWEEN effective dates
│       └── FND_USER (QA_LAST_UPDATED_BY)
│           ├── ON FU.USER_ID = QR.QA_LAST_UPDATED_BY
│           └── PER_ALL_PEOPLE_F (same pattern)

```

```

WIP_OPERATIONS_V (aliased as W01 and/or W02)
  ON WO.WIP_ENTITY_ID = EWO.WIP_ENTITY_ID
  AND WO.ORGANIZATION_ID = EWO.ORGANIZATION_ID
    └─ WIP_OP_RESOURCE_INSTANCES_V
        ON RES.WIP_ENTITY_ID = EWO.WIP_ENTITY_ID
        AND RES.OPERATION_SEQ_NUM = WO.OPERATION_SEQ_NUM
    └─ BOM_DEPARTMENTS
        ON BD.DEPARTMENT_ID = WO.DEPARTMENT_ID
        AND BD.ORGANIZATION_ID = WO.ORGANIZATION_ID

MTL_PARAMETERS
  ON MP.ORGANIZATION_ID = EWO.ORGANIZATION_ID
  (filter: MP.ORGANIZATION_CODE = 'XAU')

EAM_WORK_ORDERS_V (parts charged)
  └─ MTL_MATERIAL_TRANSACTIONS
      ON MMT.TRANSACTION_SOURCE_ID = EWO.WIP_ENTITY_ID
      AND MMT.ORGANIZATION_ID = EWO.ORGANIZATION_ID
      AND MMT.TRANSACTION_SOURCE_TYPE_ID = 5 (WIP)
        └─ MTL_SYSTEM_ITEMS_B
            ON MSI.INVENTORY_ITEM_ID = MMT.INVENTORY_ITEM_ID
            AND MSI.ORGANIZATION_ID = MMT.ORGANIZATION_ID
        └─ MTL_TRANSACTION_TYPES
            ON MTT.TRANSACTION_TYPE_ID = MMT.TRANSACTION_TYPE_ID

FND_USER (security hub – standalone from work order queries)
  └─ QA_USER_GROUP_V (QA group membership)
      ON QUG.USER_ID = FU.USER_ID
      (filter: QUG.STATUS = 'A' for active members)

      Also joins directly from QA_RESULTS:
      ON QUG.USER_ID = QR.QA_CREATED_BY
      (no FND_USER needed in the middle)

  └─ PER_ALL_PEOPLE_F
      ON PEO.PERSON_ID = FU.EMPLOYEE_ID
      AND SYSDATE BETWEEN effective dates
        └─ HR_ALL_ORGANIZATION_UNITS (FK: PER_PEOPLE_F_FK1)
            ON HOU.ORGANIZATION_ID = PEO.BUSINESS_GROUP_ID
        └─ BOM_RESOURCE_EMPLOYEES (direct via PERSON_ID)
            ON BRE.PERSON_ID = PEO.PERSON_ID
            AND SYSDATE <= BRE.EFFECTIVE_END_DATE
              └─ BOM_RESOURCES
                  ON BR.RESOURCE_ID = BRE.RESOURCE_ID
                  AND BR.ORGANIZATION_ID = BRE.ORGANIZATION_ID
                  (filter: BR.ORGANIZATION_ID = 1169)

  └─ FND_USER_RESP_GROUPS_DIRECT
      ON FUR.USER_ID = FU.USER_ID
      └─ FND_RESPONSIBILITY
          ON FR.RESPONSIBILITY_ID = FUR.RESPONSIBILITY_ID
            └─ FND_RESPONSIBILITY_TL
                ON FRT.RESPONSIBILITY_ID = FR.RESPONSIBILITY_ID
                AND FRT.LANGUAGE = 'US'
                (filter: RESPONSIBILITY_NAME = 'ICU EAM Manager - XAU')
            └─ FND_APPLICATION

```

```
ON FA.APPLICATION_ID = FR.APPLICATION_ID
(filter: APPLICATION_SHORT_NAME = 'EAM')
```

Common Filter Patterns

Filter	Value	Purpose
ORGANIZATION_ID	1169	Scope to single org
ORGANIZATION_CODE	'XAU'	Human-readable org filter via MTL_PARAMETERS
WIP_ENTITY_NAME LIKE	'AU%'	Asset-based work orders
WIP_ENTITY_NAME LIKE	'PM%'	Preventive maintenance work orders
OPERATION_SEQ_NUM	10	First/primary operation step
OPERATION_COMPLETED	'N'	Open / incomplete operations
WORK_ORDER_STATUS	'Released' / 'Cancelled'	Status filter
USER_DEFINED_STATUS_ID	3	Numeric released status
WORK_ORDER_TYPE_DISP	'DM'	Demand maintenance type
QA_CREATION_DATE >	SYSDATE - N	Rolling date window (1, 15, 31 days)
MAINTENANCE_OP_SEQ	10	QA results for primary operation
RESPONSIBILITY_NAME	'ICU EAM Manager - XAU'	EAM manager role filter
APPLICATION_SHORT_NAME	'EAM'	Scope to EAM module only
FND_USER.END_DATE	IS NULL OR > SYSDATE	Active users only
FND_RESPONSIBILITY_TL.LANGUAGE	'US'	Always required — avoids duplicate rows per language
USER_NAME / USER_ID / EMPLOYEE_ID	any of the three	Triple-OR lookup when you don't know which identifier you have: <code>WHERE fu.user name = :val OR TO_CHAR(fu.user_id) = :val OR TO_CHAR(fu.employee_id) = :val</code>
BOM_RESOURCES.ORGANIZATION_ID	1169	Scope resource lookup to XAU org
BOM_RESOURCES.RESOURCE_TYPE	2	Person-type resources only (1=Machine, 3=Space)
BOM_RESOURCES.DISABLE_DATE	IS NULL	Active resources only
BOM_RESOURCE_EMPLOYEES.EFFECTIVE_END_DATE	>= SYSDATE	Active employee-resource assignments only
QA_USER_GROUP_V.STATUS	'A'	Active group memberships only
QA_USER_GROUP_V.GROUP_NAME LIKE	'XAU PM%'	Filter QA groups by org/type prefix pattern

Diagnostic & Utility Patterns

Find all APPS tables containing a column

Use `ALL_TAB_COLUMNS` to discover tables dynamically:

```
SELECT owner, table_name
FROM all_tab_columns
WHERE column_name = 'USER_ID'
AND owner = 'APPS';
```

Probe tables for a known value (PL/SQL)

Iterate results and run dynamic SQL to find which tables actually have data for a given ID:

```

DECLARE
    v_sql    VARCHAR2(1000);
    v_count  NUMBER;
BEGIN
    FOR t IN (SELECT owner, table_name FROM all_tab_columns
              WHERE column_name = 'USER_ID' AND owner = 'APPS')
    LOOP
        v_sql := 'SELECT COUNT(*) FROM ' || t.owner || '.' || t.table_name
                || ' WHERE USER_ID = :1';
        BEGIN
            EXECUTE IMMEDIATE v_sql INTO v_count USING 10169062;
            IF v_count > 0 THEN
                DBMS_OUTPUT.PUT_LINE(t.owner || '.' || t.table_name
                                     || ' → ' || v_count || ' match(es)');
            END IF;
        EXCEPTION WHEN OTHERS THEN NULL;
        END;
    END LOOP;
END;
/

```

Bind variables (:1 USING value) are safer than string concatenation — avoids SQL injection and handles type conversion.

Check your own active session

```

SELECT sid, serial#, username, status, last_call_et
FROM v$session
WHERE username = USER;

```

Requires access to V\$SESSION . Useful for debugging long-running queries or verifying connection state.

Parts charged to work orders

Find work orders with the most part transactions in the last 30 days:

```

SELECT DISTINCT
    EWO.WIP_ENTITY_NAME,
    EWO.ASSET_NUMBER,
    COUNT(MMT.TRANSACTION_ID) AS PART_TRANSACTIONS
FROM APPS.EAM_WORK_ORDERS_V EWO
JOIN INV.MTL_MATERIAL_TRANSACTIONS MMT
    ON MMT.TRANSACTION_SOURCE_ID      = EWO.WIP_ENTITY_ID
   AND MMT.ORGANIZATION_ID            = EWO.ORGANIZATION_ID
   AND MMT.TRANSACTION_SOURCE_TYPE_ID = 5      -- 5 = WIP source
WHERE
    EWO.ORGANIZATION_ID = 1169
  AND MMT.TRANSACTION_DATE >= SYSDATE - 30
GROUP BY
    EWO.WIP_ENTITY_NAME,
    EWO.ASSET_NUMBER
ORDER BY
    PART_TRANSACTIONS DESC
FETCH FIRST 10 ROWS ONLY;

```

Full part detail for a specific work order:

```

SELECT
    EWO.WIP_ENTITY_NAME           AS WORK_ORDER,
    EWO.ASSET_NUMBER,
    EWO.ASSET_DESCRIPTION,
    EWO.WORK_ORDER_STATUS,

```

```

MTT.TRANSACTION_TYPE_NAME      AS TRANSACTION_TYPE,
TO_CHAR(MMT.TRANSACTION_DATE,
        'MM/DD/YYYY HH24:MI:SS') AS TRANSACTION_DATE,
MSI.SEGMENT1                    AS PART_NUMBER,
MSI.DESCRPTION                  AS PART_DESCRIPTION,
MMT.TRANSACTION_QUANTITY        AS QTY,
MMT.TRANSACTION_UOM             AS UOM,
MMT.SUBINVENTORY_CODE          AS FROM_SUBINVENTORY,
MMT.TRANSACTION_REFERENCE      AS REFERENCE
FROM APPS.EAM_WORK_ORDERS_V EWO
JOIN INV.MTL_MATERIAL_TRANSACTIONS MMT
  ON MMT.TRANSACTION_SOURCE_ID = EWO.WIP_ENTITY_ID
 AND MMT.ORGANIZATION_ID      = EWO.ORGANIZATION_ID
 AND MMT.TRANSACTION_SOURCE_TYPE_ID = 5
JOIN INV.MTL_SYSTEM_ITEMS_B MSI
  ON MSI.INVENTORY_ITEM_ID = MMT.INVENTORY_ITEM_ID
 AND MSI.ORGANIZATION_ID   = MMT.ORGANIZATION_ID
JOIN INV.MTL_TRANSACTION_TYPES MTT
  ON MMT.TRANSACTION_TYPE_ID = MMT.TRANSACTION_TYPE_ID
WHERE
  EWO.ORGANIZATION_ID = 1169
 AND EWO.WIP_ENTITY_NAME = 'AU14857998'
ORDER BY
  MMT.TRANSACTION_DATE DESC;

```

TRANSACTION QUANTITY is **negative for issues** (parts going out to the job) and **positive for returns**. TRANSACTION_SOURCE_TYPE_ID = 5 is the WIP filter — without it you get all inventory movements. Work order names have no hyphen (e.g. AU14857998 not AU-14857998). MACHINE SHOP as an asset number indicates a virtual asset used for shop repair work orders rather than a physical piece of equipment.

Audit trail — quick version (last touched by user, no date filter)

Useful when you have a USER_ID and want to see everything they've touched, most recent first:

```

SELECT *
FROM (
  SELECT
    'QA_RESULTS'      AS source_table,
    qr.work_order_id  AS entity_id,
    qr.qa_last_update_date AS change_date
  FROM apps.qa_results qr
  WHERE qr.qa_last_updated_by = 35566
  UNION ALL
  SELECT
    'WIP_ENTITIES',
    we.wip_entity_id,
    we.last_update_date
  FROM apps.wip_entities we
  WHERE we.last_updated_by = 35566
  UNION ALL
  SELECT
    'WIP_OPERATIONS',
    wo.wip_entity_id,
    wo.last_update_date
  FROM apps.wip_operations wo
  WHERE wo.last_updated_by = 35566
)
ORDER BY change_date DESC;

```

Replace 35566 with the target USER_ID . Add WHERE change date >= SYSDATE - N inside each subquery to limit the date range. See the full version below for CREATED_BY coverage and a date filter.

Audit trail — full version (created and updated, with date filter)

Every EBS table carries `CREATED_BY` , `LAST_UPDATED_BY` , `CREATION DATE` , `LAST_UPDATE_DATE` . Use `UNION ALL` to search across multiple tables at once. Replace `35566` with the target `USER_ID` and adjust the date filter:

```
SELECT *
FROM (
    SELECT
        'QA_RESULTS' AS source,
        qr.work_order_id AS entity_id,
        NVL(qr.qa_last_update_date, qr.qa_creation_date) AS change_date,
        CASE
            WHEN qr.qa_created_by = 35566 THEN 'CREATED'
            WHEN qr.qa_last_updated_by = 35566 THEN 'UPDATED'
        END AS action
    FROM apps.qa_results qr
    WHERE 35566 IN (qr.qa_created_by, qr.qa_last_updated_by)
    AND NVL(qr.qa_last_update_date, qr.qa_creation_date)
        >= TO_DATE('25-JAN-2026', 'DD-MON-YYYY')
    UNION ALL
    SELECT
        'WIP_ENTITIES',
        we.wip_entity_id,
        NVL(we.last_update_date, we.creation_date),
        CASE
            WHEN we.created_by = 35566 THEN 'CREATED'
            WHEN we.last_updated_by = 35566 THEN 'UPDATED'
        END
    FROM apps.wip_entities we
    WHERE 35566 IN (we.created_by, we.last_updated_by)
    AND NVL(we.last_update_date, we.creation_date)
        >= TO_DATE('25-JAN-2026', 'DD-MON-YYYY')
    UNION ALL
    SELECT
        'WIP_OPERATIONS',
        wo.wip_entity_id,
        wo.last_update_date,
        'UPDATED'
    FROM apps.wip_operations wo
    WHERE wo.last_updated_by = 35566
    AND wo.last_update_date >= TO_DATE('25-JAN-2026', 'DD-MON-YYYY')
)
ORDER BY change_date DESC;
```

Extend with more `UNION ALL` blocks for any other table — the pattern is the same. Use `NVL(last_update_date, creation_date)` when a table may have null `LAST_UPDATE_DATE` on freshly created rows.

Scripts Inventory

All scripts follow the same pattern: Python runner in `scripts/` , SQL template in `sql/` , output in `outputs/` .

Script	SQL File	Output Dir	Description
run_cancelled_wo_report.py	cancelled_work_orders_15_days.sql	outputs/cancelled_work_orders/	Cancelled work orders in the last 15 days
run_level10_dm_open.py	level10_dm_open.sql	outputs/level10_dm/	Open DM work orders at operation seq 10
run_pm_released_wo_report.py	pm_released_work_orders.sql	outputs/pm_released_work_orders/	Released PM work orders — Δ Cartesian risk with EAM_PM_SCHEDULING_RULES
run_qa_daily_results.py	qa_daily_results_24_hours.sql	outputs/qa_daily_results/	QA results in the last 24 hours
run_eam_verification.py	verify_eam_relationships.sql	outputs/eam_verification/	Verifies all table relationships and filter values

Script	SQL File	Output Dir	Description
run_person_wo_history.py	person_work_order_history.sql	outputs/person_work_order_history/	All work orders completed by a named employee. Args: --name , --days
run_top10_assets.py	top10_assets_wo_detail.sql	outputs/top10_assets/	Top 10 assets by WO volume — last 12 months, with QA detail. One Excel sheet per asset. Args: --org-code
(ad hoc)	—	—	QA user group membership for a single user: SELECT fu.user_id, fu.user_name, qug.group_name, qug.status FROM apps.fnd_user fu JOIN apps.qa_user_group_v qug ON qug.user_id = fu.user_id WHERE fu.user_name = '10169062'
(ad hoc)	—	—	Find all QA groups matching a name pattern: SELECT DISTINCT qug.group_name FROM apps.qa_user_group_v qug WHERE UPPER(qug.group_name) LIKE 'XAU PM%'

Running scripts

```
# From the project root
python scripts\run_top10_assets.py --org-code XAU
python scripts\run_person_wo_history.py --name "Smith" --days 90
python scripts\run_eam_verification.py
```

SQLcl Known Gotchas

Lessons learned running SQLcl on Windows via Python subprocess:

Issue	Symptom	Fix
Multiple substitution variables	SP2-0044 error, script aborts	Only &1 is reliable. Use __TOKEN__ in SQL and replace in Python before execution
Banner lines in CSV output	Pandas ParserError: Expected 1 fields	Call clean_csv() to strip lines before the first comma-containing line
SET VERIFY ON (default)	SQL text echoed into spool file instead of data	Add SET VERIFY OFF to every SQL script
Script runs twice	Output logged twice, second run fails	Never use str_replace to prepend — always write fresh files
SQLcl JVM startup time	Each run_query() call takes ~7-8 seconds	Batch all checks into one session or use a single SQL file with SPOOL

Python pattern for multi-parameter SQL

Since SQLcl only handles &1 reliably, inject parameters in Python:

```
def build_sql(template_path: Path, **params) -> Path:
    sql = template_path.read_text(encoding="utf-8")
    for token, value in params.items():
        sql = sql.replace(f"__{token}__", str(value))
    tmp = tempfile.NamedTemporaryFile(mode="w", suffix=".sql",
                                      delete=False, encoding="utf-8")
    tmp.write(sql)
    tmp.close()
    return Path(tmp.name)
```

Use `__ORG_CODE__`, `__DATE_FROM__`, `__NAME__` etc. as tokens in SQL templates.